

Introdución a Kubernetes

Que é a orquestración de contedores?

No desenvolvemento moderno, os **contedores** permiten empacar unha aplicación xunto co seu sistema operativo base, dependencias e configuración. Isto garante que se execute igual en calquera contorno, sexa un portátil, un servidor físico ou unha nube.

Porén, cando se traballa con **decenas ou centos de contedores** (como ocorre nas arquitecturas de microservizos), precisase unha ferramenta que os **xestione automaticamente**: cree, escale, reinicie, actualice ou elimine contedores segundo as necesidades do sistema. A isto chámase **orquestración de contedores**.

Funcionalidades principais

- Despregue automático de contedores.
- Escalado horizontal (engadir ou quitar réplicas).
- Supervisión do estado dos servizos.
- Reinicio automático ante fallos.
- Balanceo de carga interno.
- Actualizacións sen interrupción (*rolling updates*).
- Xestión de configuracións e segredos.
- Almacenamento persistente para datos.

Que é Kubernetes?

Kubernetes (tamén coñecido como **K8s**, polas 8 letras entre o "K" e o "s") é unha plataforma **de código aberto** deseñada para automatizar o despregue, escalado e operación de contedores.

Foi creada por **Google**, baseada na súa experiencia interna con Borg, e actualmente é mantida pola **Cloud Native Computing Foundation (CNCF)**. É o estándar de facto na industria para orquestrar contedores en contornos tanto locais como en nube.

Obxectivos principais

- Automatizar o despregue e a xestión de contedores.
- Asegurar alta dispoñibilidade e tolerancia a fallos.
- Facilitar o despregue continuo (*CI/CD*).
- Simplificar o escalado horizontal e vertical.
- Garantir portabilidade entre diferentes provedores cloud.

Características principais de Kubernetes

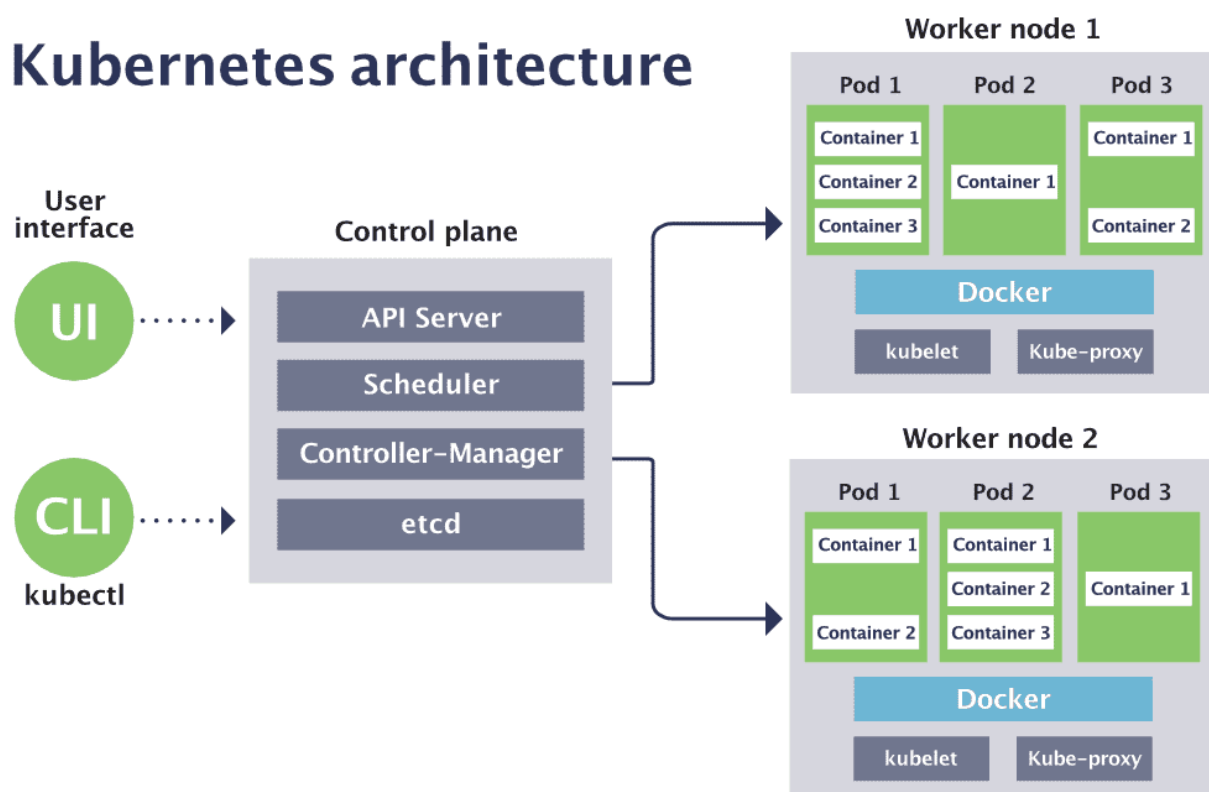
- **Despregue automatizado** de aplicacións en contedores.
- **Auto-reparación**: reinicia contedores que fallan ou se perden.
- **Actualizacións sen interrupcións** e posibilidade de *rollback*.
- **Balanceo de carga** e resolución DNS interna.

- **Xestión de almacenamento persistente** (PVs e PVCs).
- **Configuración e segredos** centralizados (ConfigMaps e Secrets).
- **Infraestrutura declarativa** mediante ficheiros YAML.
- **Extensibilidade**: admite controladores e operadores personalizados.
- **Portabilidade**: pode executarse en local (Minikube, Kind) ou na nube (GKE, EKS, AKS).

Arquitectura de Kubernetes

Kubernetes ten unha arquitectura **cliente-servidor** dividida en dous compoñentes principais: o **Control Plane** (ou plano de control) e os **nós de traballo** (*worker nodes*).

Kubernetes architecture



Control Plane (nó mestre)

É o cerebro de Kubernetes: xestiona o estado desexado do clúster e toma decisións sobre onde e como se executan os contedores.

- **kube-apiserver** → expón a API REST de Kubernetes; é o punto central de comunicación.
- **etcd** → almacén chave-valor que garda todo o estado do clúster.
- **kube-scheduler** → decide en que nodo executar cada Pod.
- **controller-manager** → supervisa os recursos e aplica accións automáticas (crear, eliminar, reiniciar).
- **cloud-controller-manager** → integra o clúster co provedor cloud (opcional).

Worker Nodes (nós de traballo)

Son as máquinas (físicas ou virtuais) onde realmente se executan os contedores.

- **kubelet** → axente que comunica co Control Plane e garante que os contedores funcionen segundo o plan.
- **kube-proxy** → xestiona o tráfico de rede cara aos Pods e o balanceo.
- **Container Runtime** → software que executa os contedores (ex: containerd, CRI-O, Docker).

Recursos fundamentais de Kubernetes

Kubernetes organízase en torno a obxectos que describen o estado desexado do clúster: aplicacións despregadas, recursos de rede, almacenamento, seguridade, etc.

A seguinte táboa resume os principais **recursos fundamentais**:

Elemento	Descrición	Exemplo de uso
Pod	Unidade mínima de execución. Pode conter un ou varios contedores.	Un microservizo web con Nginx.
Deployment	Xestiona réplicas de Pods, actualizacións e <i>rollbacks</i> .	Escalar un servizo de API REST.
StatefulSet	Mantén identidade e almacenamento fixo para cada Pod.	Despregue dunha base de datos.
DaemonSet	Executa un Pod en cada nodo do clúster.	Axente de monitorización.
Job / CronJob	Execución única ou programada de tarefas.	Xerar informes diarios.
Service	Expón un conxunto de Pods cunha IP estable.	Balancear tráfico entre réplicas.
Ingress	Regras HTTP/HTTPS externas para acceder a servizos.	Expor unha aplicación web pública.
Namespace	Aíllamento lóxico de recursos.	Separar dev/test/prod.
ConfigMap	Gardar configuracións non sensibles.	Variables de contorno dunha app.
Secret	Gardar datos sensibles (tokens, contrasinais).	Credenciais de base de datos.
Volume / PVC	Almacenamento persistente para datos.	Gardar logs ou ficheiros.
StorageClass	Define a política de aprovisionamento dinámico de volumes.	Crear PVC automaticamente.
RBAC (Roles, Bindings)	Controla permisos e acceso a recursos.	Limitar accións dun usuario.
Autoscaler (HPA/VPA)	Escala pods horizontal ou verticalmente.	Adaptar a carga de traballo.
NetworkPolicy	Define regras de tráfico entre Pods.	Illar servizos críticos.

Pods

O **Pod** é a unidade mínima de execución en Kubernetes. Contén un ou varios contedores que comparten rede e volumes.

Tipos de Pods (por uso)

Tipo	Descrición	Exemplo
Independente	Creado manualmente, sen controladores.	Proba puntual dun contedor.
Controlado	Xestionado por Deployment, StatefulSet, etc.	Servizo web replicado.
Multi-container	Varios contedores colaboran no mesmo Pod.	Patrón <i>sidecar</i> (proxy + app).
Efémero	Engadido temporalmente para depurar.	<code>kubect1 debug pod</code>

Exemplo básico:

```
apiVersion: v1
kind: Pod
metadata:
  name: exemplo-pod
spec:
  containers:
    - name: nginx
      image: nginx:1.27-alpine
      ports:
        - containerPort: 80
```

Controladores de Pods (*Controllers*)

Os *Controllers* xestionan a creación, actualización e dispoñibilidade dos Pods. Permiten despregues declarativos e tolerancia a fallos.

Tipo	Características	Uso típico
Deployment	Xestiona réplicas e actualizacións continuas.	Aplicacións sen estado.
StatefulSet	Mantén identidade e almacenamento persistente.	BBDD, Kafka, Redis...
DaemonSet	Un Pod por nodo, ideal para servizos do sistema.	Axentes de logs ou métricas.
Job	Executa unha tarefa única e remata.	Procesado batch.
CronJob	Lanza Jobs de forma periódica.	Backups, tarefas programadas.

Exemplo de Deployment:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
    spec:
      containers:
        - name: nginx
          image: nginx:1.27-alpine
          ports:
            - containerPort: 80

```

Exemplo de CronJob:

```

apiVersion: batch/v1
kind: CronJob
metadata:
  name: limpeza-logs
spec:
  schedule: "0 2 * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: limpeza
              image: busybox
              command: ["sh", "-c", "rm -rf /var/log/*.log"]
          restartPolicy: OnFailure

```

Services

Os **Services** proporcionan un punto de acceso estable a un conxunto de Pods, que poden recrearse ou cambiar de IP.

Tipo	Descrición	Uso típico
ClusterIP	IP interna accesible só dentro do clúster.	Comunicación entre microservizos.

Tipo	Descripción	Uso típico
NodePort	Abre un puerto no nodo accesible dende fóra.	Probas locais con Minikube.
LoadBalancer	Crea un balanceador externo (cloud).	Aplicacións en produción.
ExternalName	Redirixe a un dominio externo.	Servizos externos.

Exemplo con NodePort:

```

apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 31000
  type: NodePort

```

Ingress e NetworkPolicy

Ingress

Define regras HTTP/HTTPS externas para acceder aos *Services*.

Exemplo básico:

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
    - host: app.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web-service
                port:
                  number: 80

```

NetworkPolicy

As **NetworkPolicies** controlan o tráfico de rede entre Pods.

Tipo	Descrición	Exemplo
Ingress	Define que tráfico pode entrar a un Pod.	Só aceptar conexións dun namespace concreto.
Egress	Define a saída de tráfico desde un Pod.	Limitar acceso a Internet.

Exemplo:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: permitir-app
spec:
  podSelector:
    matchLabels:
      app: backend
  ingress:
    - from:
      - podSelector:
          matchLabels:
            app: frontend
```

ConfigMaps e Secrets

Permiten **parametrizar aplicacións** sen modificar imaxes.

Tipo	Contido	Persistencia	Exemplo
ConfigMap	Configuración non sensible.	Non cifrado.	Nome de aplicación, modo debug.
Secret	Datos sensibles (tokens, contrasinais).	Codificado base64.	Credenciais, claves API.

Exemplo de ConfigMap:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  modo: produccion
  port: "8080"
```

Exemplo de Secret:

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  usuario: YWRtaW4=
  contrasinal: c2VjcmV0
```

Montaxe como variables de contorno:

```
envFrom:
- configMapRef:
  name: app-config
- secretRef:
  name: db-secret
```

Volumes, PVC e StorageClass

Os contedores son efémeros, polo que Kubernetes emprega **Volumes** para persistir datos.

Tipos de Volumes

Tipo	Descrición	Persistencia
emptyDir	Directorio temporal, elimínase co Pod.	✗
hostPath	Directorio do nodo anfitrión.	⚠ (dependente do nodo)
configMap / secret	Volume xerado dende eses recursos.	⚠
persistentVolumeClaim (PVC)	Volume provisionado dinámicamente.	☑

Exemplo de PVC:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: datos-app
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

Exemplo de uso nun Deployment:

```
volumeMounts:
  - name: datos
    mountPath: /usr/share/nginx/html
volumes:
  - name: datos
    persistentVolumeClaim:
      claimName: datos-app
```

StorageClass

Define o tipo de almacenamento e a política de aprovisionamento:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: rapido
provisioner: kubernetes.io/aws-ebs
parameters:
  type: gp2
reclaimPolicy: Delete
volumeBindingMode: Immediate
```

Namespaces

Os **Namespaces** permiten dividir o clúster en espazos lóxicos illados.

É útil para separar entornos (*dev*, *test*, *prod*) ou equipos.

```
apiVersion: v1
kind: Namespace
metadata:
  name: dev
```

Comandos útiles:

```
kubectl get namespaces
kubectl get pods -n dev
```

RBAC (Roles e permisos)

O sistema **RBAC** (*Role-Based Access Control*) controla quen pode facer que accións sobre que recursos.

Recurso	Función	Alcance
Role	Define permisos nun namespace.	Local
ClusterRole	Define permisos globais.	Clúster
RoleBinding	Asocia Role a usuario/grupo.	Local
ClusterRoleBinding	Asocia ClusterRole.	Global
ServiceAccount	Identidade usada polos Pods.	Namespace

Exemplo de Role e RoleBinding:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: lector-pods
  namespace: dev
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: vinculo-lector
  namespace: dev
subjects:
  - kind: User
    name: ana
roleRef:
  kind: Role
  name: lector-pods
  apiGroup: rbac.authorization.k8s.io
```

Autoscalers

Os **Autoscalers** permiten adaptar automaticamente os recursos segundo a carga.

Tipo	Descrición	Axuste
HorizontalPodAutoscaler (HPA)	Escala número de réplicas.	Horizontal
VerticalPodAutoscaler (VPA)	Axusta CPU/memoria dun Pod.	Vertical

Exemplo de HPA:

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  minReplicas: 2
  maxReplicas: 5
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 75

```

Esquema xeral de relación entre recursos

```

[User / DevOps]
|
[kubectl / API Server]
|
[Namespace: dev/test/prod]
|
[Deployment / StatefulSet / DaemonSet]
|
[Pods] <---> [ConfigMap / Secret / PVC]
|
[Service] <---> [Ingress / NetworkPolicy]
|
[Cluster networking + Autoscaling]

```

Configuración con YAML

Kubernetes funciona de forma **declarativa**: definimos o estado desexado do sistema e o clúster encárgase de facer que o estado real coincida con el. Ao crear un obxecto en YAML estamos dicindo: “isto é o que quero que teña o meu clúster”. ([Guía oficial de obxectos en Kubernetes](#))

Estrutura básica dun manifesto

Todos os manifestos comparten a seguinte estrutura común:

```

apiVersion: <grupo/versión> # Exemplo: v1, apps/v1, networking.k8s.io/v1
kind: <Recurso> # Exemplo: Pod, Deployment, Service, Ingress...
metadata:
  name: <nome>
  namespace: <opcional> # Se non se especifica, usa "default"
  labels: # Etiquetas opcionais para organización
    app: <etiqueta>
spec:
  # Contido específico do recurso

```

Que significa cada apartado

- **apiVersion:** Indica o grupo de API e versión deste recurso.
- **kind:** Tipo de obxecto que creas.
- **metadata:** Información de identificación: nome, etiquetas, etc.
- **spec:** A configuración que desexas aplicar (o estado "querido").
- (Adicionalmente os obxectos teñen un campo **status**, pero ese é xestionado polo sistema e non o escribes ti). ([Conceptos sobre obxectos en Kubernetes](#))

Parámetros típicos por recurso

Pod e Containers (base para todos os recursos)

Os **Pods** son a unidade mínima de execución en Kubernetes. Cada Pod pode conter un ou varios **containers** que comparten rede e volumes. A maioría dos recursos (Deployment, Job, StatefulSet...) crean Pods segundo a súa propia configuración.

Campos habituais en **spec** dun Pod

Campo	Descrición	Exemplo
containers[]	Lista de contedores principais.	image: nginx:1.27-alpine
initContainers[]	Contedores que se executan antes dos principais.	Migracións de base de datos.
volumes[]	Volumes dispoñibles no Pod.	PVC, ConfigMap, Secret...
restartPolicy	Política de reinicio (Always , OnFailure , Never).	Always (por defecto).
nodeSelector / affinity	Controla onde se executa o Pod.	Afinidade por etiquetas de nodo.
serviceAccountName	Conta de servizo asociada (RBAC).	ansible-service

Campos típicos de **containers[]**

Campo	Descrición	Exemplo
-------	------------	---------

Campo	Descrición	Exemplo
<code>name</code>	Nome interno do contedor.	<code>web</code>
<code>image</code>	Imaxe a despregar.	<code>nginx:alpine</code>
<code>ports[]</code>	Portos expostos polo contedor.	<code>containerPort: 80</code>
<code>env / envFrom</code>	Variables de contorno.	<code>envFrom: configMapRef:</code>
<code>resources.requests/limits</code>	Reservas de CPU/RAM.	<code>cpu: "100m", memory: "128Mi"</code>
<code>volumeMounts[]</code>	Montaxes de volumes no contedor.	<code>mountPath: /data</code>
<code>livenessProbe / readinessProbe</code>	Verificacións de saúde.	HTTP GET / en porto 80

Exemplo mínimo dun Pod:

```

apiVersion: v1
kind: Pod
metadata:
  name: exemplo-pod
spec:
  containers:
    - name: web
      image: nginx:1.27-alpine
      ports:
        - containerPort: 80

```

Deployment (apps/v1)

Un **Deployment** xestiona a creación e actualización dun grupo de Pods idénticos. É o recurso máis usado para despregar aplicacións “sen estado” (*stateless*).

Campos habituais

Campo	Descrición	Exemplo
<code>replicas</code>	Número de réplicas do Pod.	<code>3</code>
<code>selector.matchLabels</code>	Debe coincidir coas etiquetas do template.	<code>app: webapp</code>
<code>template.metadata.labels</code>	Etiquetas dos Pods creados.	<code>app: webapp</code>
<code>template.spec.containers</code>	Definición dos contedores (ver sección anterior).	<code>image: nginx:alpine</code>

Campo	Descripción	Ejemplo
<code>strategy.type</code>	Estrategia de actualización (<code>RollingUpdate</code> , <code>Recreate</code>).	<code>RollingUpdate</code>
<code>minReadySeconds</code>	Tempo mínimo antes de considerar un Pod listo.	<code>5</code>

Ejemplo completo de Deployment:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
    spec:
      containers:
        - name: web
          image: nginx:latest
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "250m"
              memory: "256Mi"
          readinessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 3
            periodSeconds: 5

```

Service (v1)

- `type`: tipo de servicio (`ClusterIP`, `NodePort`, `LoadBalancer`, `ExternalName`).
- `selector`: etiquetas que indican que Pods serán "atendidos" por el servicio.
- `ports[]`: define `port`, `targetPort`, e para tipos `NodePort` o `nodePort`.
- Documentación completa de [Services en Kubernetes](#).

Ingress (networking.k8s.io/v1)

- `rules[]`: definición de host/path -> backend service.
- `tls[]`: se van usar HTTPS, certificados, etc.
- As anotacións (annotations) dependen do controlador de Ingress que uses.
- Documentación oficial de [Ingress](#).

ConfigMap & Secret (v1)

- **ConfigMap**: usado para configuracións non sensibles. Campo `data`: con pares clave/valor.
- **Secret**: usado para datos sensibles (tokens, contrasinais). Os valores están base64 codificados.
- Montaxes típicas: como variables de contorno (envFrom) ou como volume.
- Referencia: [ConfigMaps e Secrets](#)

Volumes / PVC / StorageClass

- **PersistentVolumeClaim (PVC)**: `accessModes` (ex: ReadWriteOnce), `resources.requests.storage`, `storageClassName` se o usas.
- **StorageClass**: define como provisionar volumes dinámicamente.
- [Xestión declarativa de obxectos e almacenamento](#)

Namespaces

- Usados para illar recursos dentro dun clúster. Os nomes deben ser únicos dentro dun namespace, pero poden repetirse en diferentes namespaces.
- [Documentación sobre Namespaces](#)

Plantillas mínimas para comezar

1) Deployment + Service

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: web
          image: nginx:1.27-alpine
          ports:
            - containerPort: 80
          resources:
```

```

    requests:
      cpu: "100m"
      memory: "128Mi"
    limits:
      cpu: "250m"
      memory: "256Mi"
  readinessProbe:
    httpGet:
      path: /
      port: 80
    initialDelaySeconds: 3
    periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: myapp
spec:
  type: ClusterIP
  selector:
    app: myapp
  ports:
    - port: 80
      targetPort: 80

```

2) Ingress para o serviço anterior

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: myapp-ingress
spec:
  rules:
    - host: myapp.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: myapp
                port:
                  number: 80

```

3) ConfigMap + uso no Deployment

```

apiVersion: v1
kind: ConfigMap
metadata:

```



```

  name: app-config
data:
  MODE: "production"
  PORT: "8080"
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-config
spec:
  selector:
    matchLabels:
      app: myapp-config
  template:
    metadata:
      labels:
        app: myapp-config
    spec:
      containers:
        - name: web
          image: nginx:1.27-alpine
          envFrom:
            - configMapRef:
                name: app-config

```

4) Secret + montaxe como env

```

apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  DB_USER: YWRtaW4=          # "admin" en base64
  DB_PASS: c2VjcmV0         # "secret" en base64
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  selector:
    matchLabels:
      app: api
  template:
    metadata:
      labels:
        app: api
    spec:
      containers:
        - name: api

```

```
image: ghcr.io/example/api:1.0
envFrom:
  - secretRef:
      name: db-secret
```

5) PVC + uso nun Deployment

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data
spec:
  accessModes: [ ReadWriteOnce ]
  resources:
    requests:
      storage: 1Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: files
spec:
  selector:
    matchLabels:
      app: files
  template:
    metadata:
      labels:
        app: files
    spec:
      containers:
        - name: web
          image: nginx:1.27-alpine
          volumeMounts:
            - name: data
              mountPath: /usr/share/nginx/html
      volumes:
        - name: data
          persistentVolumeClaim:
            claimName: data
```

Boas práticas

- Usa `kubectl explain <recurso>` para ver todos os campos a usar.
[Referencia de kubectl explain](#)
- Gárdalos en **vários ficheiros** nun directorio `manifests/` e aplica con:

```
kubectl apply -f ./manifests/
```

Isto simplifica a xestión e separación de recursos.

[Xestión declarativa de obxectos](#)

- Asegúrate de que labels e selectors coincidan correctamente entre Deployment e Service.
- Configura `resources.requests` e `resources.limits` sempre que sexa produción para evitar saturacións.
- Define probes (`readinessProbe`, `livenessProbe`) para garantir saúde dos contedores.
- Se tes varios contornos (dev/test/prod), considera usar **Kustomize** (overlays) para reutilizar base.
- Respecta os formatos e nomes recomendados de etiquetas para facer máis amigable a xestión: [etiquetas recomendadas](#)

Lista de verificación antes de aplicar un manifesto

- ☒ Está correctamente especificado `apiVersion` e `kind`.
- ☒ O recurso `metadata.name` é único dentro do namespace.
- ☒ Se aplica un recurso que depende doutro (ex: Service depende de Deployment), o Deployment debe etiquetar correctamente antes de aplicar Service.
- ☒ Os campos obrigatorios en `spec` están presentes (consultar con `kubectl explain`).
- ☒ Labels e selectors coinciden.
- ☒ Se se usan montaxes de almacenamento, o PVC ou volume está provisionado.
- ☒ Se usas `type: LoadBalancer` ou ingress, entendes que depende da infraestrutura.
- ☒ Aplicas desde un directorio ordenado e usas control de versións nos teus manifestos.

Referencias oficiais para profundizar

- [Conceptos sobre obxectos en Kubernetes](#)
- [kubectl explain – Documentación da CLI](#)
- [Declarative management of Kubernetes objects using configuration files](#)
- [Namespaces – illamento lóxico de recursos](#)
- [Recommended Labels – etiquetas recomendadas](#)

Escalado e actualizacións

Escalado manual

```
kubectl scale deployment webapp --replicas=5
```

Escalado automático

```
kubectl autoscale deployment webapp --min=2 --max=10 --cpu-percent=70
```

Actualizaci3n

```
kubectl set image deployment/webapp web=nginx:1.25
kubectl rollout status deployment/webapp
kubectl rollout undo deployment/webapp
```

Monitorizaci3n e depuraci3n

Comandos 3tiles:

```
kubectl get pods -o wide
kubectl logs <pod>
kubectl exec -it <pod> -- sh
kubectl top pods
kubectl describe pod <pod>
```

En Minikube:

```
minikube addons enable metrics-server
```

Ferramentas externas: **Prometheus**, **Grafana**, **Lens**, **K9s**.

Resumo final

Elemento	Funci3n principal
Pod	Unidade m3nima de execu33n
Deployment	Control de r3plicas e versi3ns
Service	Comunica33n estable entre Pods
Ingress	Acceso HTTP/HTTPS externo
ConfigMap / Secret	Configuraci3n e segredos
Volume / PVC	Almacenamento persistente
HPA	Escalado autom3tico

Elemento	Función principal
Probe	Control de saúde e dispoñibilidade

Autor: Adrián Blanco

Centro: CE CIFP A Carballeira

Data: Novembro 2025